

OWASP WebGoat e WebWolf

Seminário de Segurança da Informação

Giuliano Marcus Bianco

Luís Renato Freitas de Almeida

Nicolas Arthur Raulino Oliveira

Instituto Federal de Santa Catarina

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Sumário

- 1 Introdução
- 2 A OWASP
- 3 OWASP WebGoat e WebWolf
- 4 Demonstração
- 5 Conclusão

Introdução

Objetivos desse seminário

- Demonstrar e explicar vulnerabilidades de forma prática a partir das aplicações OWASP WebGoat e OWASP WebWolf.
- Identificar os principais métodos de ataques.
- Identificar boas práticas de segurança do desenvolvimento de aplicações.
- Demonstrar a importância da segurança em aplicações Web.

Contextualização

- O crescimento das aplicações web trouxe novos desafios de segurança.
- A OWASP promove boas práticas e ferramentas para mitigar vulnerabilidades.
- WebGoat e WebWolf são plataformas didáticas para aprendizado prático de segurança.

A OWASP

O que é a OWASP



OWASP (*Open Worldwide Application Security Project*) é uma **Organização sem fins lucrativos** que trabalha para melhorar a segurança de software. É uma comunidade aberta dedicada a habilitar organizações a conceber, desenvolver, adquirir, operar e manter aplicações confiáveis.

A OWASP Foundation

- **Projetos Open Source** liderados pela comunidade, incluindo código, documentação e padrões.
- **Dezenas de milhares de membros** ativos.
- Todos os projetos, ferramentas, documentos e fóruns são **livres e abertos** a qualquer pessoa interessada.
- Fundada em 1º de dezembro de 2001, incorporada como organização sem fins lucrativos nos Estados Unidos em 21 de abril de 2004.

OWASP WebGoat e WebWolf

O que é o WebGoat

- Aplicação web **deliberadamente insegura** para testar vulnerabilidades comumente encontradas em aplicações Java.
- Resolve o problema de aprender segurança web sem precisar de aplicações reais vulneráveis (como lojas online ou bancos).
- Permite que profissionais de segurança testem ferramentas em uma plataforma conhecida vulnerável.
- Fornece um ambiente **seguro e legal** para prática e aprendizado.

Objetivo e Metodologia do WebGoat

- **Objetivo principal:** criar um ambiente de ensino interativo para segurança de aplicações web.
- **Metodologia de aprendizado em três passos:**
 - 1 **Explicar a vulnerabilidade:** conceitos teóricos antes da prática.
 - 2 **Aprender fazendo:** exercícios práticos para compreender o funcionamento.
 - 3 **Explicar mitigação:** técnicas de proteção e boas práticas.

O que é o WebWolf

- Aplicação web **separada** que simula a máquina de um atacante.
- Permite distinguir claramente entre o papel do **atacante** e o papel do **usuário** no website.
- Criado após feedback de workshops onde não havia distinção clara entre esses papéis.
- Funciona como um software auxiliar do WebGoat.

Funcionalidades do WebWolf

- **Hospedar arquivos:** permite fazer upload de arquivos necessários para download durante exercícios.
- **Cliente de e-mail:** simula o envio de e-mails para testar vulnerabilidades relacionadas.
- **Página de destino para requisições:** serve como landing page para receber chamadas de dentro de exercícios, fornecendo informações completas sobre a requisição (semelhante ao `netcat`¹).

¹<https://nmap.org/ncat/>

Integração WebGoat e WebWolf

- WebGoat representa o **website atacado** (aplicação vulnerável).
- WebWolf representa o **ambiente do atacante** (máquina do atacante).
- Juntos, simulam cenários reais de ataque de forma didática e controlada.
- A distinção clara entre os papéis facilita o aprendizado prático de segurança web.

Ferramentas Necessárias

- **WebGoat**²: aplicação web J2EE executada no servidor Apache Tomcat.
- **WebWolf**: aplicação complementar que simula o ambiente do atacante.
- **Proxy web**: ferramenta para interceptar e analisar requisições HTTP (ex: **OWASP ZAP**³, Burp Suite).

²<https://owasp.org/www-project-webgoat/>

³<https://www.zaproxy.org/>

Avisos de Segurança

Importante

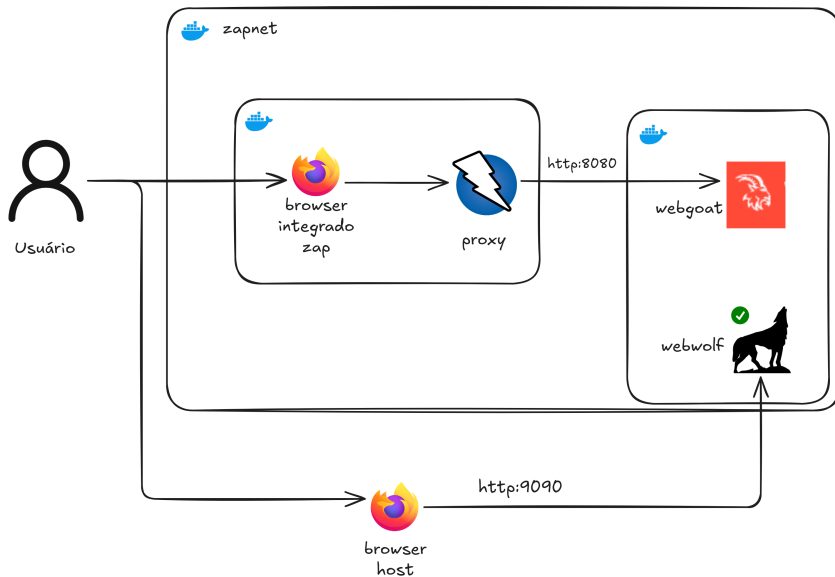
- **Não** execute o WebGoat em um IP externamente visível; ele é vulnerável a ataques.
- **Não** execute o WebGoat nem mesmo em uma sub-rede IP reservada; sempre use `localhost`.
- Durante a execução, sua máquina estará **extremamente vulnerável** a ataques.
- **Desconecte-se da Internet** enquanto estiver usando o programa.
- A configuração padrão do WebGoat vincula-se ao `localhost` para minimizar a exposição.

Demonstração

Ambiente de Teste

- Ambiente containerizado usando **Docker Compose** para facilitar a configuração e execução.
- **Rede isolada** (zapnet) conectando todos os containers.
- **WebGoat**: aplicação vulnerável na porta 8080.
- **WebWolf**: incluído na imagem WebGoat, disponível na porta 9090 (acessível via localhost).
- **OWASP ZAP**: proxy que atua como *Man-in-the-Middle* (MITM), posicionando-se entre o navegador e o servidor para interceptar e analisar requisições HTTP/HTTPS.
- **Volumes persistentes** para manter dados entre execuções. (caso alguém queira usar o container para praticar em casa)

Ambiente de Teste



Vulnerabilidades JWT: Contexto

- **JSON Web Token (JWT):** padrão para autenticação e autorização em aplicações web modernas.
- Estrutura: `header.payload.signature`
- Tokens são assinados digitalmente para garantir integridade e autenticidade.
- Vulnerabilidades comuns surgem de:
 - Falta de validação adequada da assinatura
 - Algoritmos de assinatura fracos ou mal implementados
 - Manipulação do payload sem verificação

Ataques que Demonstraremos

1 Decodificação de JWT

- Análise da estrutura do token
- Extração de informações sensíveis do payload
- Risco: exposição de dados em tokens não criptografados

2 Manipulação de JWT

- Alteração do payload para elevar privilégios
- Exploração de validação inadequada de assinatura
- Objetivo: tornar-se administrador e executar ações privilegiadas

Mitigações para essas vulnerabilidades do JWT

- **Decodificação:** não colocar dados sensíveis no payload (ou usar JWE para criptografar)
- **Manipulação:** sempre validar a assinatura e rejeitar tokens com algoritmos "none"

SQL Injection: Contexto

- **SQL Injection (SQLi):** ataque que explora falhas na validação de entradas que são usadas em consultas SQL.
- Ocorre quando dados fornecidos pelo usuário são concatenados diretamente na query.
- Permite que o atacante:
 - Leia, modifique ou delete dados do banco
 - Bypasse autenticação
 - Execute comandos perigosos no banco de dados
- Afeta aplicações sem sanitização adequada ou sem uso de queries preparadas.

Ataques que Demonstraremos

1 Personificação com ZAP para Extração de Dados Confidenciais

- Enviar uma Requisição API REST de método POST com Query SQL
- Demonstrar roubo de informações do banco de dados com manipuladores (MITM)

2 Comprometer Confidencialidade com Injeção String SQL

- Inserção de payloads com ' OR '1'='1'–
- Exploração de consultas vulneráveis para listar tabelas e dados sensíveis
- Demonstração do potencial de vazamento completo da base

Mitigações para SQL Injection

- **Uso de Prepared Statements (queries parametrizadas)**
 - Evitam concatenação direta de dados na query
- **Validação e Sanitização de Entrada**
 - Rejeitar caracteres inesperados e validar formatos
- **Princípio do Menor Privilégio**
 - Contas do banco não devem ter permissões além do necessário
- **ORMs e Query Builders seguros**
 - Evitam a construção manual de consultas inseguras

Spoofing de Cookie de Autenticação: Contexto

- **Spoofing de Cookie de Autenticação:** Tipo de ataque de personificação que explora cookies de sessão mal implementados.
- Ocorre quando um atacante obtém ou **falsifica um cookie de sessão válido** para se passar por outro usuário.
- O atacante explora:
 - Falhas na **criação/codificação** do cookie (ex: uso de algoritmos reversíveis ou previsíveis).
 - Falhas na **segurança da transmissão** (ex: ausência de HTTPS ou flags Secure/HttpOnly).
 - Falhas de **Validação** (ex: o servidor confia cegamente no valor do cookie sem checar a integridade).
- Permite ao atacante **burlar o mecanismo de autenticação** e obter acesso não autorizado.

Análise e Impacto do Spoofing de Cookie

1 Análise da Previsibilidade do Cookie

- Foco na descoberta do **algoritmo de geração** (ex: codificação previsível como Base64 ou concatenação simples).
- O objetivo educacional é entender como a previsibilidade permite a **reversão e a falsificação** para um novo usuário.

2 Comprometimento da Confidencialidade e Integridade

- Demonstração do potencial de **acesso total à conta** do usuário legítimo (*personificação*).
- Análise do impacto de **ações não autorizadas** (leitura de dados, modificação de configurações) realizadas pelo atacante.

Mitigações para Spoofing de Cookie

■ Tokens de Sessão Criptograficamente Seguros

- Usar tokens de sessão longos, aleatórios e **criptograficamente fortes** (ex: UUIDs).

■ Assinatura Digital dos Cookies

- Utilizar HMAC (Hash-based Message Authentication Code) para **validar a integridade** do cookie no servidor, impedindo modificações.

■ Uso de Flags de Cookie Seguras

- HttpOnly: Previne acesso ao cookie via JavaScript (defesa contra XSS).
- Secure: Garante que o cookie só seja enviado em conexões **HTTPS**.
- SameSite: Ajuda a prevenir CSRF e vazamento de cookies entre sites.

■ Vincular Sessão ao Endereço IP/User-Agent

- Revalidar a sessão e expirar o cookie se houver uma **mudança drástica no contexto** do usuário.

Conclusão

Considerações Finais

- WebGoat e WebWolf permitem aprendizado prático e seguro de vulnerabilidades.
- Ajudam a compreender como proteger aplicações contra ataques reais.
- São ferramentas essenciais para o ensino de segurança da informação.

Referências I



ANDERSON, Ross J. **Security Engineering: A Guide to Building Dependable Distributed Systems**. 3. ed. Hoboken: Wiley, 2020. ISBN 978-1-119-64278-7.



CYCUBIX. **WebGoat | Web Application Security Essentials**. 2025. Acesso em: 17 nov. 2025. Disponível em: <<https://docs.cycubix.com/application-security-series/web-application-security-essentials/webgoat-8>>.



FOUNDATION, OWASP. **OWASP Top 10 – The Ten Most Critical Web Application Security Risks**. 2021. Acesso em: 12 nov. 2025. Disponível em: <<https://owasp.org/Top10/>>.



_____. **OWASP WebGoat: A deliberately insecure web application maintained by OWASP for educational purposes**. 2025. Acesso em: 12 nov. 2025. Disponível em: <<https://owasp.org/www-project-webgoat/>>.



_____. **OWASP WebWolf: Companion application for WebGoat to simulate attacker environment**. 2025. Acesso em: 12 nov. 2025. Disponível em: <<https://owasp.org/www-project-webgoat/>>.

Referências II



FOUNDATION, OWASP. **The Open Web Application Security Project (OWASP)**. 2025. Acesso em: 12 nov. 2025. Disponível em: <<https://owasp.org>>.



MALYALA, Lalithya. **Careers in Web app Testing**. Ago. 2020. Acesso em: 12 nov. 2025. Disponível em: <https://owasp.org/www-chapter-vancouver/assets/presentations/2020-08_careers_in_web_app_testing.pdf>.



STALLINGS, William. **Cryptography and Network Security: Principles and Practice**. 8. ed. Boston: Pearson Education, 2020. ISBN 978-0-13-444428-4.